SWE Take-Home Exercise (Full-Stack)

Overview

Build a small full-stack web application using the stack below. The goal is not feature volume, but correctness, clarity, and thoughtful tradeoffs.

Time expectation: ~3 hours
Clean, well-explained, correct code is preferred over feature-rich complexity.

You will implement a **Base Implementation** (required), and then choose **ONE** of two focused extensions:

- Variant A — Backend-Focused (caching + correctness)
- Variant B — Frontend/API-Focused (client orchestration + API contracts)

Tech Stack (Required)

- TypeScript
- Next.js (App Router preferred)
- React + TailwindCSS
- Postgres
- Prisma
- Google OAuth
- OpenAI API

You may use documentation, Google, and AI tools. However, you must be able to clearly explain everything you submit.

Base Implementation (Required for Everyone)

Product: Movie Memory

1. Authentication

- Visiting `/` while unauthenticated shows a landing page with:
    - "Sign in with Google"
- After successful Google login:
    - First-time users are redirected to onboarding.
    - Returning users go to the dashboard.

---

2. Onboarding (First-Time Only)

Prompt the user for their **favorite movie**.

Requirements:

- Server-side validation (trim input, min/max length).
- Store the movie in Postgres.
- Associate it with the authenticated user.

---

3. Dashboard

The dashboard must display:

- User name (from Google)
- User email (from Google)
- User photo (if available)
- Favorite movie (from DB)
- Logout button

If logged out and navigating to `/dashboard`, redirect to login.

---

4. Basic Fact Generation (Simple Version)

Display a fun fact about the user's favorite movie using OpenAI.

For the base implementation:

- It is acceptable to generate a fact on each request.
- Store generated facts in the database.

(Your chosen variant will add more sophistication.)

Choose ONE Variant

After completing the base implementation, choose **either Variant A or Variant B**.

In your README, clearly state which variant you chose and why.

Variant A — Backend-Focused (Caching & Correctness)

Enhance fact generation with backend logic and correctness guarantees.

Requirements

1. 60-Second Cache Window

Modify fact generation so that:

- If the most recent fact for this user + movie is **less than 60 seconds old**, return the cached fact.
- If older than 60 seconds, generate a new fact, store it, and return it.

2. Burst / Idempotency Protection

Prevent multiple new facts from being generated simultaneously if:

- The user refreshes quickly
- Multiple tabs are open

Acceptable approaches (choose one):

- DB transaction guard
- "Generation in progress" flag
- Unique constraint per user per 60-second window
- Simple in-memory lock (must document limitation)

3. Failure Handling

If OpenAI fails (timeout / error):

- Return the most recent cached fact if one exists
- Otherwise return a user-friendly error

4. Minimal Backend Tests

Add tests for:

- 60-second cache logic
- Authorization (user cannot fetch another user's facts)

Evaluation Emphasis (Variant A)

- Data modeling quality
- Correct cache window implementation
- Concurrency reasoning
- Safe OpenAI integration
- Backend correctness

Variant B — Frontend/API-Focused (Client Orchestration)

Enhance the application with stronger API contracts and frontend state management.

Requirements

1. Typed API Layer

Implement API endpoints such as:

- `GET /api/me`
- `PUT /api/me/movie`
- `GET /api/fact`

Create a typed client wrapper (e.g., `lib/api.ts`) that:

- Handles requests consistently
- Parses typed responses

- Normalizes error handling

2. Edit Movie Flow (Inline)

On the dashboard:

- Allow inline editing of favorite movie
- Include Save + Cancel
- Use optimistic UI update
- Revert if server fails

3. Client-Side Caching

Implement simple client caching:

- Cache last fact with timestamp
- If within 30 seconds, reuse cached fact unless user explicitly requests a new one
- Invalidate cache when movie changes

You may use React state, SWR, React Query, or your own approach. Explain your choice.

4. Minimal Frontend/API Tests

Add tests for:

- API client error handling (e.g., 401 or 500)
- Movie edit flow behavior

Evaluation Emphasis (Variant B)

- API contract clarity
- Typed client design
- Loading / error / empty states
- Cache invalidation correctness
- Frontend state reasoning

Security & Correctness Requirements (Applies to All)

You must:

- Validate movie input server-side
- Ensure users cannot access or modify other users' data
- Avoid exposing secrets to the client
- Gracefully handle missing Google photo or name

Deliverables

Provide:

1. GitHub repository
2. README including:
    - Setup instructions
    - Required environment variables
    - Database migration steps
    - Architecture overview (1–2 pages)
    - Which variant you chose and why
    - Key tradeoffs you made
    - What you would improve with 2 more hours
3. Short note on AI usage (bullets describing how you used it)

Optional but encouraged:

- 3–5 minute walkthrough video explaining key implementation decisions

---

Evaluation Criteria (General)

We evaluate on:

1. Auth flow correctness
2. Data modeling quality
3. Security and authorization
4. Error handling
5. Code clarity and structure
6. Ability to explain decisions
7. Depth of implementation in chosen variant

Important

You should be able to clearly explain:

- Why you designed the schema the way you did
- Your caching or client-state strategy (depending on variant)
- How you handled tradeoffs and failure cases

Clean, thoughtful engineering beats over-engineering.